Introduction to ROS2 (Packages, Nodes, Communication, Parameters, Launch Files & LED Exercise)

Introduction to Robotics Lab

Konstantinos Asimakopoulos *k_asimakopoulos@ac.upatras.gr*
Lampros Printzios *printzios_lampros@ac.upatras.gr*

Department of Electrical and Computer Engineering
University of Patras

October 12, 2026

Table of Contents

1. Introduction and First Steps
2. Packages
3. Nodes
4. ROS2 Communication System
5. Parameters
6. Launch files
7. Inspecting ROS2 Graph activity
8. Luna the Rover Exercise
9. LED Exercise
10. References

Introduction and First Steps - What is ROS?

1. **ROS (Robot Operating System)** is an open-source, flexible **development** framework, **visualization** software, and **packaging** system, that simplifies the development of complex and distributed robotic systems.
2. ROS is not a traditional operating system; rather, it acts as **middleware** that enables communication between hardware (motors, sensors, cameras), algorithms, and applications, across multiple platforms and programming languages (e.g., Python, C++).
3. ROS is released in distributions (**distros**), like the ones below:
   - ROS 1 **Noetic** $\rightarrow$ Ubuntu 20.04
   - ROS 2 **Foxy**, **Galactic** $\rightarrow$ Ubuntu 20.04
   - ROS 2 **Humble** $\rightarrow$ Ubuntu 22.04
   - ROS 2 **Jazzy** $\rightarrow$ Ubuntu 24.04

We will use ROS2 only, and mainly Python, for the code we run in the lab.

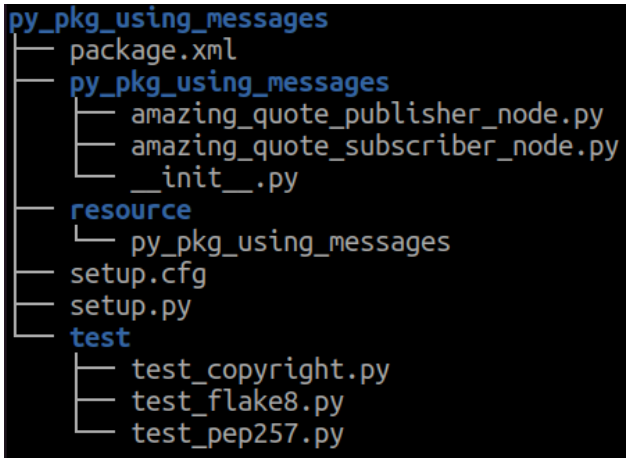
Packages - What is a Package?

1. A **Package** is the basic **unit of organization** in ROS2.
2. It groups together everything needed to share, build, and run a piece of functionality, making the software modular, reusable, and easy to maintain.
3. It usually contains:
   - **Source code** → executable nodes, libraries, or interface definitions (`msg`, `srv`, `action`)
   - **Configuration files** → parameters, launch files, and resource files
   - **Build system files** → metadata for building, packaging, installing, and resolving dependencies (e.g. `package.xml`, `setup.py`, `setup.cfg`, `CMakeLists.txt`)

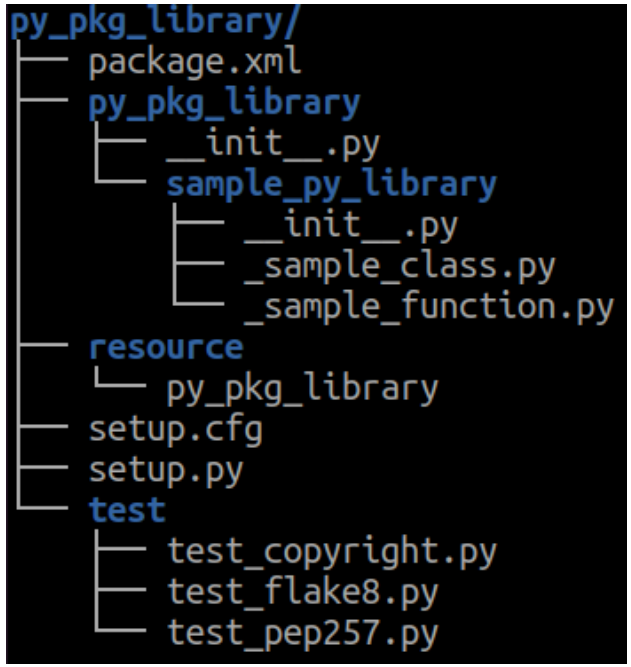
ROS2 Package Functionalities

```
ros2 pkg -h
```

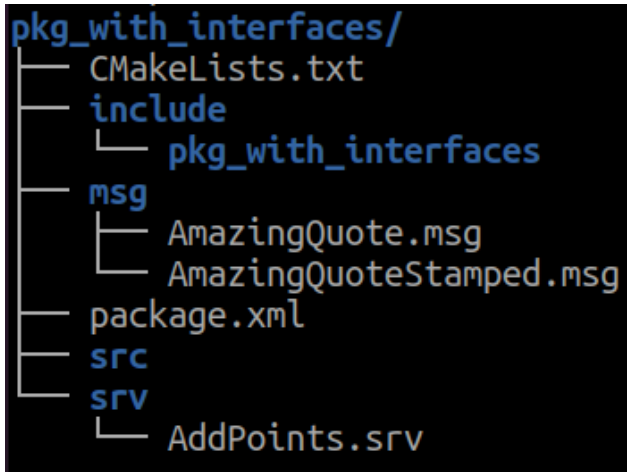
Packages - Basic Package Structures



Package with Publisher and Subscriber Nodes



Library Package



Interfaces Package

Each Python package directory contains an inner folder that defines the main *Python Module* (with the same name) where the source code is located, and some other build and packaging metadata, like `package.xml`, `setup.py` and `setup.cfg`. For the Interfaces Package, we are mainly interested in `msg` and `srv` (optionally `action`) folders, while the build metadata are `package.xml` and `CMakeLists.txt`.

Packages - Creating & building a Package

We create a source folder `src/` in our `<ros2_ws>` workspace and enter it:

```
mkdir ~/<ros2_ws>/src && cd ~/<ros2_ws>/src
```

To create a package (`ament_cmake` for C++, instead of `ament_python`):

```
ros2 pkg create <pkg_name> --build-type
ament_python --dependencies rclpy
<other_pkg>
```

We build from the `<ros2_ws>` folder with **`colcon build`**, which creates `build/`, `install/` and `log/`. After building, always type **`source install/setup.bash`** to make the changes visible to your terminal.

`ros2 pkg create -h` lists the creation options.

```
<?xml version="1.0"?> <?xml-model
href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>py_pkg_using_messages</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="TODO">lampros</maintainer>
  <license>TODO: License declaration</license>

  <depend>rclpy</depend>
  <depend>pkg_with_interfaces/</depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

`package.xml` of an `ament_python` package; in red, lines you may need to add before building (mainly dependencies)

Packages - Creating & building a Package - setup.py

The **setup.py** file exists in ament_python packages, but not in ament_cmake ones (similarly with setup.cfg file).

```
from setuptools import setup

package_name = 'py_pkg_node'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='lampros',
    maintainer_email='[redacted]',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'sample_py_node = py_pkg_node.sample_py_node:main',
            'print_forever_node = py_pkg_node.print_forever_node:main'
        ],
    },
)
```

simple setup.py file

Important!

We should always add the executable nodes in the console_scripts list (marked with red color).

Nodes - What is a Node?

1. **Nodes** are **executable units** / processes, doing some computation.
2. **Each device or algorithm** in our robotic system can be a **distinct node**.
3. They are **decoupled** (no shared memory) but **talk to each other by using interfaces**, via **publish-subscribe** or **request-response**.

ROS2 Node Functionalities

```
ros2 node -h
```

create_timer

```
create_timer(timer_period_sec, callback)
```

```python
import rclpy
from rclpy.node import Node

...
class PrintForever(Node):
    def __init__(self):
        super().__init__("print_forever")
        timer_period: float = 1.0
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.print_count: int = 0
    def timer_callback(self):
        self.get_logger().info(f"Printed {self.print_count} times.")
        self.print_count += 1

def main(args = None):
    try:
        rclpy.init(args = args)
        print_forever_node = PrintForever()
        rclpy.spin(print_forever_node)
    except KeyboardInterrupt:
        pass
    except Exception as e:
        print(e)

if __name__ == "__main__":
    main()
```

Simple ROS2 Python Node

Nodes - What is a Node?

Let's assume that the node `print_forever_node.py` belongs to the package `py_pkg_node`. To make the node executable, we need to add it to the `console_scripts` list in the `setup.py` file as it is shown right below (`<print_forever_node>` is chosen as the name of our node).

At the end of `setup.py` file

```
entry_points = {'console_scripts':  ['print_forever_node =
py_pkg_node.print_forever_node:main']}
```

Now, after we build the modified package, we can type the command `ros2 run py_pkg_node print_forever_node` to run the node, or, in the general case:

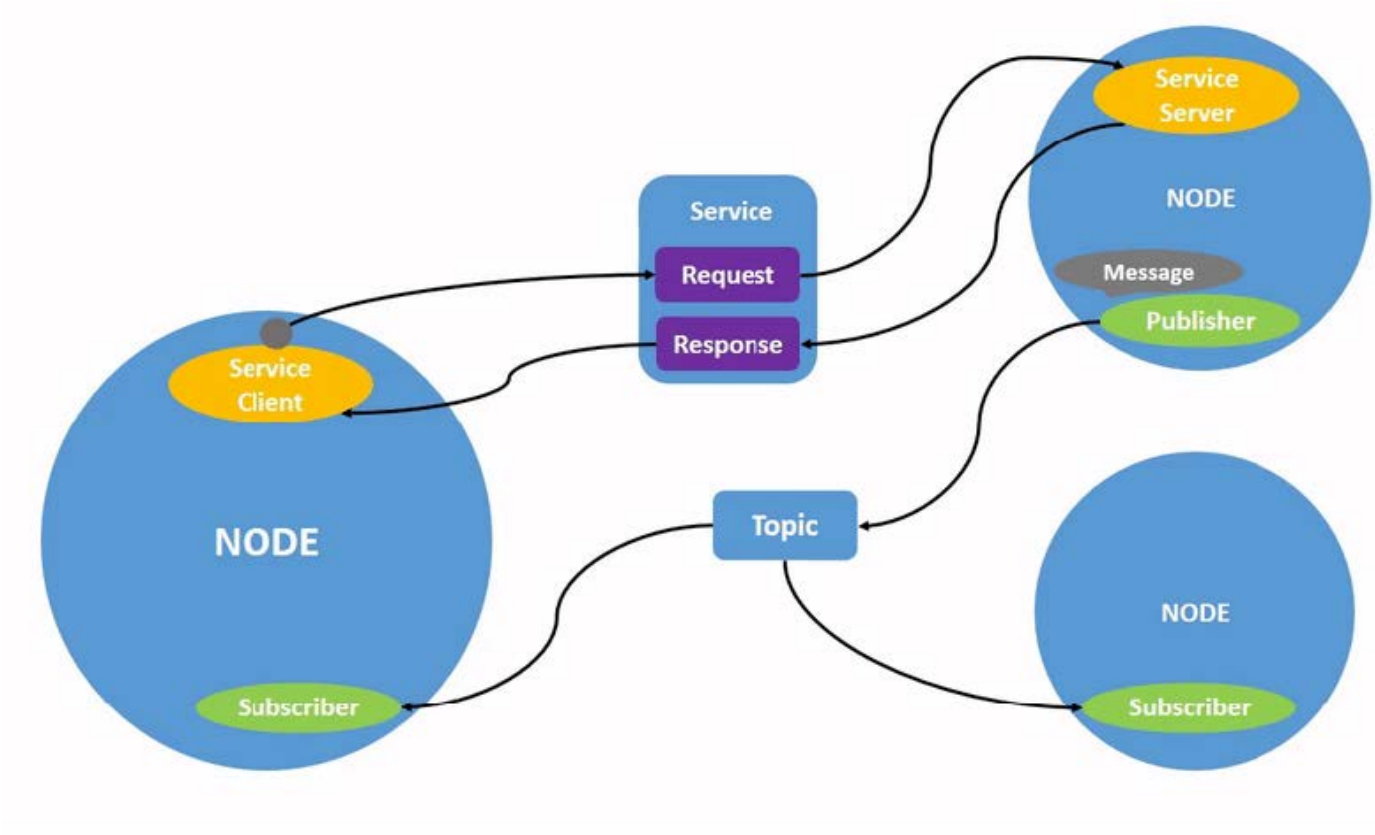
Command to run a node

```
ros2 run <pkg_name> <node_name>
```

To see the currently available (running) nodes, type `ros2 node list`.

ROS2 Communication System - General Communication Scheme

ROS2 General Communication Scheme



Here we see a basic ROS 2 communication scheme consisting of three nodes. Each node can act as a **publisher**, **subscriber**, **service server**, or **service client**. They communicate using simple messages over topics, and request–response message pairs through services.

ROS2 Communication System - Interfaces

Interfaces are files written in the ROS2 Interface Description Language (IDL). We can use them to communicate between nodes, that can even be written in different languages (Python, C++, ...). They are of 3 types:

- **Messages** → `.msg` files (simplest interface form)
- **Services** → `.srv` files (request & response messages)
- **Actions** → `.action` files (mixture of messages and services)

Some very well-known interface packages are **`geometry_msgs`** and **`sensor_msgs`**. Of course, we can write our own interface packages.

ROS2 Interface Functionalities

```
ros2 interface -h
```

ros2 interface show geometry_msgs/msg/Point

```
# This contains the position
of a point in free space
float64 x
float64 y
float64 z
```

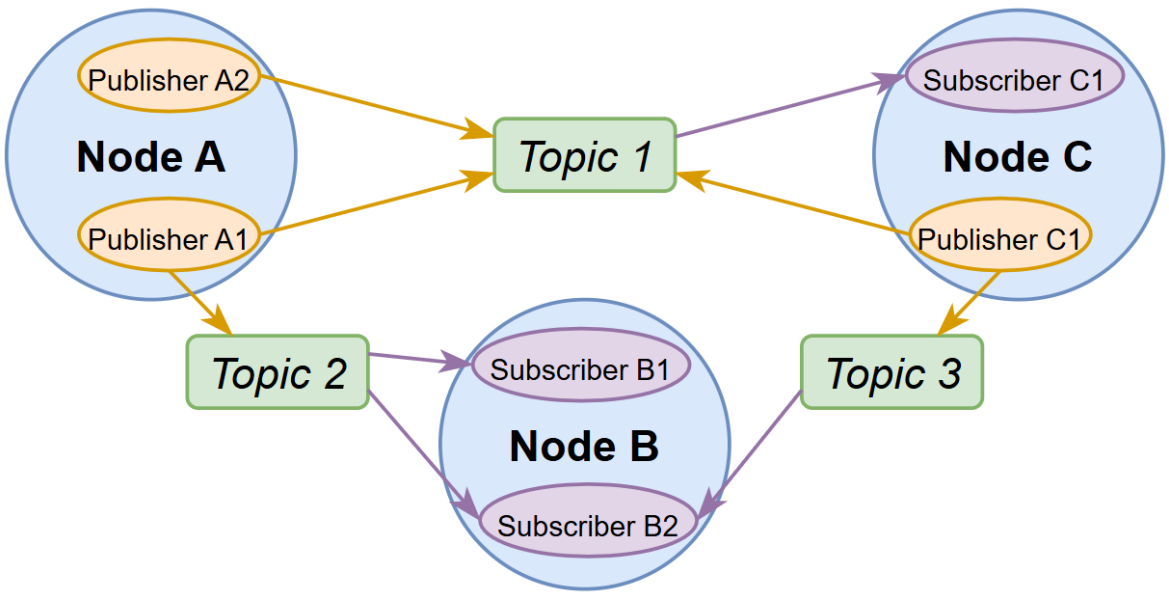
ROS2 Communication System - Interfaces - Messages

1. **Messages** (`.msg`) are data structures, the building blocks of communication over **topics**, **services**, and **actions**.
2. A message consists of **typed fields** (e.g. `int32`, `float64`, `bool`, `string`, `time`).
3. **Topics** are named communication channels (**buses**) that use **exclusively messages** as payload.

ros2 interface show geometry_msgs/msg/Pose

```
# A pose in free space, composed
of position and orientation
Point position
Quaternion orientation
```

`ros2 topic list`, `ros2 topic -h`



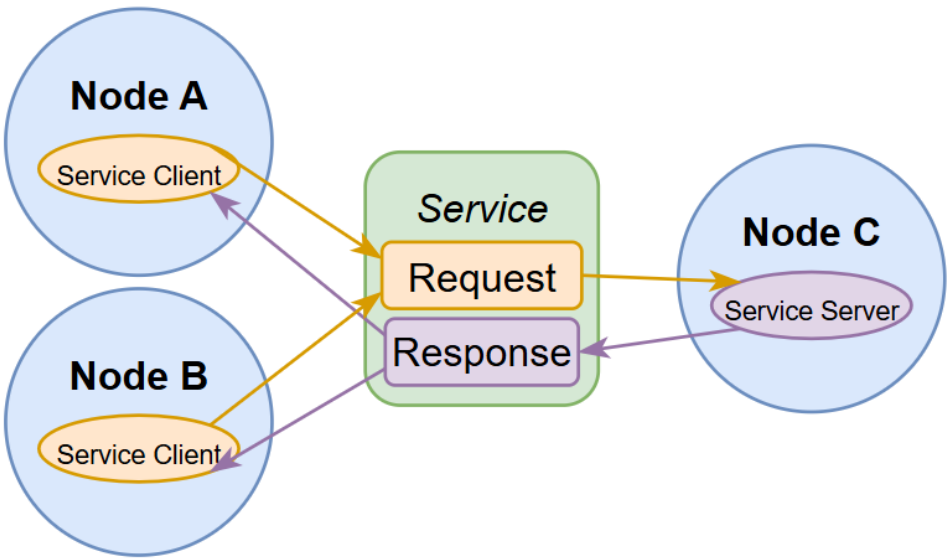
Topics, Publishers & Subscribers: Example communication scheme

ROS2 Communication System - Interfaces - Services

1. A **Service** is similar to a topic, as both enable data exchange between nodes, but a topic is *one-way* and *continuous* (publish - subscribe), while a service is ***two-way*** and ***on-demand*** (request - response).
2. Each service has a ***single*** **Service Server**, that processes a **request** and sends back a **response**, and can have ***multiple*** **Service Clients**.

AddPoints.srv (example)

```
geometry_msgs/Point a          # Req.
geometry_msgs/Point b
---
geometry_msgs/Point result # Resp.
```



Services, Servers & Clients: Example communication scheme

ROS2 Communication System - Interfaces - Actions

1. An **Action** is a **communication mechanism** similar to a service, as both provide *two-way, on-demand* interaction.
2. It differs from a service in the sense that it is designed for ***long-running tasks***, where the client may need **feedback** during execution, and the option to **cancel** the task.
3. Each action is defined by three message structures:
   - **Goal** (sent by the client to the server)
   - **Result** (sent by the server after completion)
   - **Feedback** (sent by the server while executing)

ros2 interface show nav2_msgs/action/FollowWaypoints

```
geometry_msgs/PoseStamped[] poses                    # Goal
---
int32[] missed_waypoints                             # Result
---
uint32 current_waypoint                              # Feedback
```

ROS2 Communication System - Publisher & Subscriber

create_publisher

`(msg_type, topic, qos_profile)`

```python
import rclpy
from rclpy.node import Node
from pkg_with_interfaces.msg import AmazingQuote

...
class AmazingQuotePublisherNode(Node):
    def __init__(self):
        super().__init__("amazing_quote_publisher_node")
        self.amazing_quote_publisher = self.create_publisher(
            msg_type = AmazingQuote,
            topic = "/amazing_quote",
            qos_profile = 1)
        timer_period: float = 1.0
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.incremental_id: int = 0
    def timer_callback(self):
        amazing_quote = AmazingQuote()
        amazing_quote.id = self.incremental_id
        amazing_quote.quote = "Use the force, Pikachu!"
        amazing_quote.philosopher_name = "Uncle Ben"
        self.amazing_quote_publisher.publish(amazing_quote)
        self.incremental_id += 1

def main(args = None):
    # Written in the standard try-except scheme for Nodes
    ...

if __name__ == "__main__":
    main()
```

Simple ROS2 Python Publisher

create_subscription

`(msg_type, topic, callback, qos_profile)`

```python
import rclpy
from rclpy.node import Node
from pkg_with_interfaces.msg import AmazingQuote

...
class AmazingQuoteSubscriberNode(Node):
    def __init__(self):
        super().__init__("amazing_quote_subscriber_node")
        self.amazing_quote_subscriber = self.create_subscription(
            msg_type = AmazingQuote,
            topic = "/amazing_quote",
            callback = self.amazing_quote_subscriber_callback,
            qos_profile = 1)
    def amazing_quote_subscriber_callback(self, msg: AmazingQuote):
        self.get_logger().info(f"\" I have received the most amazing of quotes.
        It says '{msg.quote}'. It was thought by the genious -- {msg.
        philosopher_name}. This latest quote had the id = {msg.id}. \"")

def main(args = None):
    # Written in the standard try-except scheme for Nodes
    ...

if __name__ == "__main__":
    main()
```

Simple ROS2 Python Subscriber

Important!

A message from another package makes that package a dependency: add it to `package.xml`.

ROS2 Communication System - Service Server & Client

create_service

`(srv_type, srv_name, callback)`

```python
import rclpy
from rclpy.node import Node
from pkg_with_interfaces.srv import AddPoints

class AddPointsServiceServerNode(Node):
    def __init__(self):
        super().__init__("add_points_service_server")
        self.service_server = self.create_service(
            srv_type = AddPoints,
            srv_name = "/add_points",
            callback = self.add_points_service_callback
        )
        self.service_server_call_count: int = 0
    def add_points_service_callback(self,
                                    request: AddPoints.Request,
                                    response: AddPoints.Response
                                    ) -> AddPoints.Response:
        response.result.x = request.a.x + request.b.x
        response.result.y = request.a.y + request.b.y
        response.result.z = request.a.z + request.b.z
        return response

def main(args = None):
    # Written in the standard try-except scheme for Nodes
    ...

if __name__ == "__main__":
    main()
```

Simple ROS2 Python Service Server

create_client

`(srv_type, srv_name)`

```python
import random
import rclpy
from rclpy.node import Node
from rclpy.task import Future
from pkg_with_interfaces.srv import AddPoints

class AddPointsServiceClientNode(Node):
    def __init__(self):
        super().__init__("add_points_service_client")
        self.service_client = self.create_client(
            srv_type = AddPoints,
            srv_name = "/add_points"
        )
        while not self.service_client.wait_for_service(timeout_sec = 1.0):
            self.get_logger().info(f"Service {self.service_client.srv_name} not available,
            waiting ...")
        self.future: Future = None
        self.timer = self.create_timer(0.5, self.timer_callback)
    def timer_callback(self):
        request = AddPoints.Request()
        request.a.x = random.uniform(0, 100)
        request.a.y = random.uniform(0, 100)
        request.a.z = random.uniform(0, 100)
        request.b.x = random.uniform(0, 100)
        request.b.y = random.uniform(0, 100)
        request.b.z = random.uniform(0, 100)
        self.get_logger().info(f"Trying to add ({request.a.x}, {request.a.y}, {request.a.z}) and
        ({request.b.x}, {request.b.y}, {request.b.z}) ...")
        if self.future is not None and not self.future.done():
            self.future.cancel()
            self.get_logger().warn("Service Future cancelled. The Node took too long to process the
            service call. Is the Service Server still alive?")
        self.future = self.service_client.call_async(request)
        self.future.add_done_callback(self.process_response)
    def process_response(self, future: Future):
        response = future.result()
        if response is not None:
            self.get_logger().info(f"The result was {(response.result.x, response.result.y,
            response.result.z)}")
        else:
            self.get_logger().info("The response was None.")

def main(args = None):
    # Written in the standard try-except scheme for Nodes
    ...

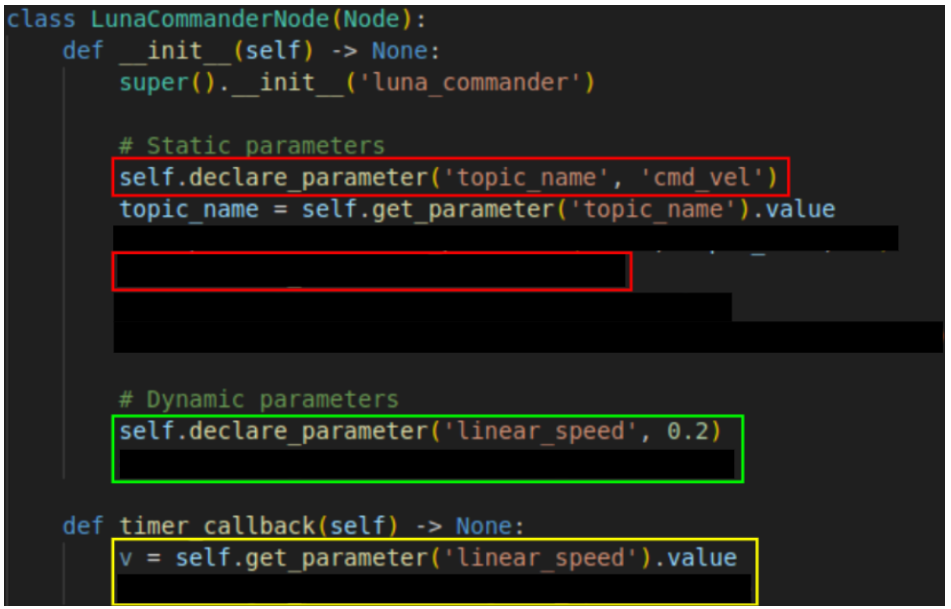
if __name__ == "__main__":
    main()
```

Simple ROS2 Python Service Client

Parameters - What are parameters?

1. **Parameters** are key - value pairs owned by a node, making it more ***reusable*** and ***configurable***: we **change its behavior** without modifying the source code (algorithm ***tuning***, ***environment-specific configuration***).
2. They can be **static** (read once at startup) or **dynamic** (read periodically and updated at runtime via callbacks).
3. They are set **programmatically** (rclpy, rclcpp), through **YAML files** or from the **CLI**.

```
declare_parameter(name, value)
get_parameter(name).value
```



parameters declared for the `luna_commander` node

static: `period`, `topic_name`. *dynamic*: `linear_speed`, `angular_speed` (read in the `timer_callback`). `period` could be dynamic with `add_on_set_parameters_callback` (next slide).

Parameters - Manage parameters

CLI (while the node runs): `ros2 param list`; `get` works on static & dynamic parameters, `set` only on dynamic ones.

```
ros2 param get /<node> <param>
ros2 param set /<node> <param> <value>
ros2 param dump /<node> --output-dir
<dir>
ros2 param load /<node> <file>
```

Reacting to changes (**validate** values, **adjust timers**, **log**): register `add_on_set_parameters_callback`; it gets the parameters (`name`, `type_`, `value`) and returns a `SetParametersResult` to accept or reject the change.

YAML files store and reload the parameters of a node (good practice: a `config` folder in the package):

- top-level key: the `<node_name>`
- parameters under the key `ros__parameters`
- *integers*, *floats*, *strings*, *booleans*, *lists*, ...

```yaml
/luna_commander:
  ros__parameters:
    angular_speed: 0.7
    linear_speed: 1.4
    period: 2.8
    topic_name: "luna_cmd_vel"
```

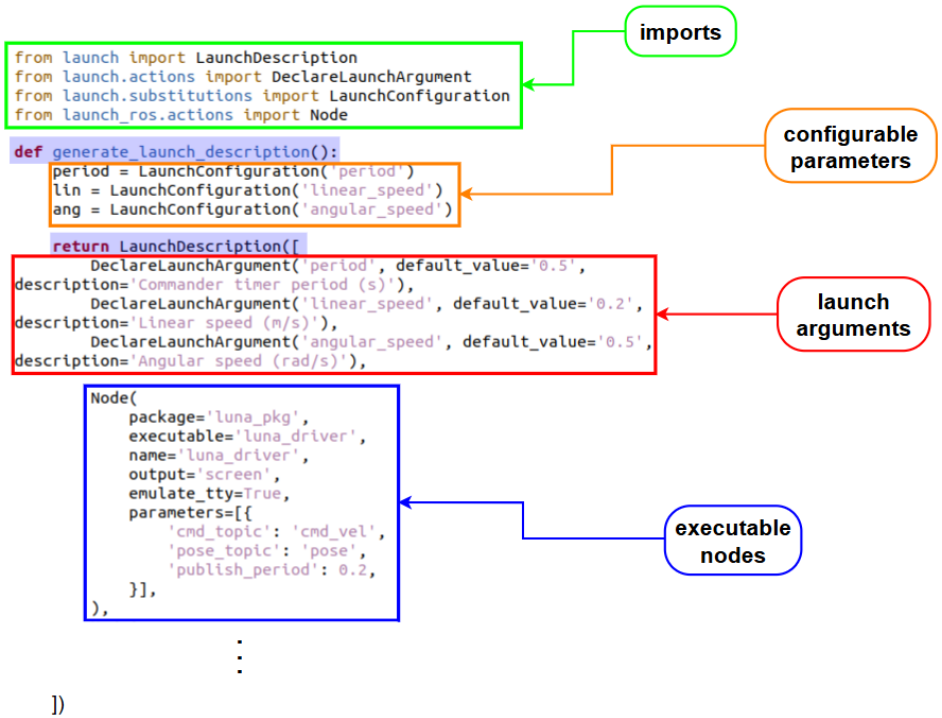
```python
from rclpy.parameter import Parameter
from rcl_interfaces.msg import SetParametersResult
def parameters_callback(self, params: List[Parameter]):
    for param in params:
        ...  # what to do for every changed parameter
    return SetParametersResult(successful = True, reason = "Correct!")
```

Launch files - What are launch files?

1. A **launch file** is a ***Python script*** that ***starts*** and ***configures*** multiple **ROS2 nodes** together (written in any language).
2. It provides **automation** (several nodes, one command), **configuration** (parameters, remappings, arguments) and **reusability** across experiments.
3. Good practice: save them in a `launch` folder of the package.

ROS2 Launch Functionalities

```
ros2 launch -h
```



launch file simple example

Launch files - Running a launch file

1. To execute a launch file we use:

Run a launch file

```
ros2 launch <package_name> <launch_file_name>
<launch_arguments>
```

2. This will start all the nodes defined in the file, with the specified parameters and remappings.

```
[INFO] [launch]: All log files can be found below /home/lampros/.ros/log/2025-10-16-17-50-19-532405-Ubuntu2004Lampros-9748
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [luna_driver-1]: process started with pid [9750]
[INFO] [luna_commander-2]: process started with pid [9752]
[luna_driver-1] [INFO] [1760626220.305305951] [luna_driver]: Pose -> x: 0.00, y: 0.00,  theta: 0.00
[luna_driver-1] [INFO] [1760626220.475235388] [luna_driver]: Pose -> x: 0.00, y: 0.00,  theta: 0.00
[luna_commander-2] [INFO] [1760626220.597868508] [luna_commander]: Command: turn left at 0.50 rad/s
[luna_driver-1] [INFO] [1760626220.680939669] [luna_driver]: Pose -> x: 0.00, y: 0.00,  theta: 0.10
[luna_driver-1] [INFO] [1760626220.874652606] [luna_driver]: Pose -> x: 0.00, y: 0.00,  theta: 0.10
[luna_commander-2] [INFO] [1760626221.073236390] [luna_commander]: Command: forward at 0.20 m/s
```

Here we launch the nodes /luna_driver and /luna_commander with:
`ros2 launch luna_package luna_package_launch.py`

We could also launch like this to change some parameters:
`ros2 launch luna_package luna_package_launch.py period:=2.0 linear_speed:=10.0 angular_speed:=1.0`

Inspecting ROS2 Graph activity - Parameters

Output a list of available parameters

```
ros2 param list
```

```
/luna_commander:
  angular_speed
  linear_speed
  period
  topic_name
  use_sim_time
```

```
/luna_driver:
  cmd_topic
  pose_topic
  publish_period
  use_sim_time
```

Get parameter

```
ros2 param get /<node_name>
    <parameter_name>
```

```
ros2 param get /luna_driver
    publish_period
Double value is: 2.0
```

Set parameter

```
ros2 param set /<node_name>
  <parameter_name> <value>
```

```
ros2 param set /luna_commander
    linear_speed 0.7
Set parameter successful
```

Inspecting ROS2 Graph activity - Parameters

Dump the parameters of a node to a yaml file

```
ros2 param dump
  /<node_name>
```

```
ros2 param dump /luna_commander
    --output-dir config/
```

```
Saving to:  config/luna_commander.yaml
```



```
cat config/luna_commander.yaml
/luna_commander:
  ros__parameters:
    angular_speed: 0.5
    linear_speed: 0.2
    period: 2.0
    topic_name: cmd_vel
    use_sim_time: false
```

Load parameter file for a node

```
ros2 param load
  /<node_name>
<parameter_file>
```

```
ros2 param load /luna_commander
  config/luna_commander.yaml
```

```
Set parameter angular_speed successful
Set parameter linear_speed successful
Set parameter period successful
Set parameter topic_name successful
Set parameter use_sim_time successful
```

Inspecting ROS2 Graph activity - Nodes

Output a list of available nodes

```
ros2 node list
```

```
/luna_commander
/luna_driver
```

Output information about a node

```
ros2 node info
  /<node_name>
```

```
ros2 node info /luna_driver
```

```
/luna_driver
  Subscribers:
    /cmd_vel: geometry_msgs/msg/Twist
  Publishers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /pose: geometry_msgs/msg/Pose2D
    /rosout: rcl_interfaces/msg/Log
  Service Servers:
    /luna_driver/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /luna_driver/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /luna_driver/get_parameters: rcl_interfaces/srv/GetParameters
    /luna_driver/list_parameters: rcl_interfaces/srv/ListParameters
    /luna_driver/set_parameters: rcl_interfaces/srv/SetParameters
    /luna_driver/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
```

Inspecting ROS2 Graph activity - Topics

Output a list of available topics

```
ros2 topic list
```

```
/cmd_vel
/parameter_events
/pose
/rosout
```

Output information about a topic

```
ros2 topic info
 /<topic_name>
```

ros2 topic info /pose

```
Type: geometry_msgs/msg/Pose2D
Publisher count: 1
Subscription count: 0
```

Output messages from a topic

```
ros2 topic echo
 /<topic_name>
```

ros2 topic echo /pose

```
x: -10.983736057091209
y: -2.591432153770138
theta: 65.0
---
```

Inspecting ROS2 Graph activity - Topics

Publish a message to a topic

ros2 topic pub -r <Hz> /<topic_name> <message_type> <values>

```
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.
Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(
x=0.0, y=0.0, z=1.0))
```

```
ros2 topic pub -r 2 /cmd_vel geometry_msgs/msg/Twist "linear:  x:
  0.5, y:  0.0, z:  0.0, angular:  x:  0.0, y:  0.0, z:  1.0"
```

Print a topic's type

ros2 topic type /<topic_name>

```
ros2 topic type /cmd_vel
geometry_msgs/msg/Twist
```

Output a list of available topics of a given type

ros2 topic find <topic_type>

```
ros2 topic find
geometry_msgs/msg/Twist
/cmd_vel
```

Inspecting ROS2 Graph activity - Services

Output a list of available services

ros2 service list

/luna_commander/get_parameters, /luna_commander/set_parameters, /luna_commander/list_parameters, ...

Output a service's type

ros2 service type /<service_name>

ros2 service type /luna_commander/get_parameters → rcl_interfaces/srv/GetParameters

Output a list of available services of a given type

ros2 service find <service_type>

ros2 service find rcl_interfaces/srv/GetParameters

Show the request & response structure of a service type

ros2 interface show <service_type>

ros2 interface show rcl_interfaces/srv/GetParameters

Inspecting ROS2 Graph activity - Services

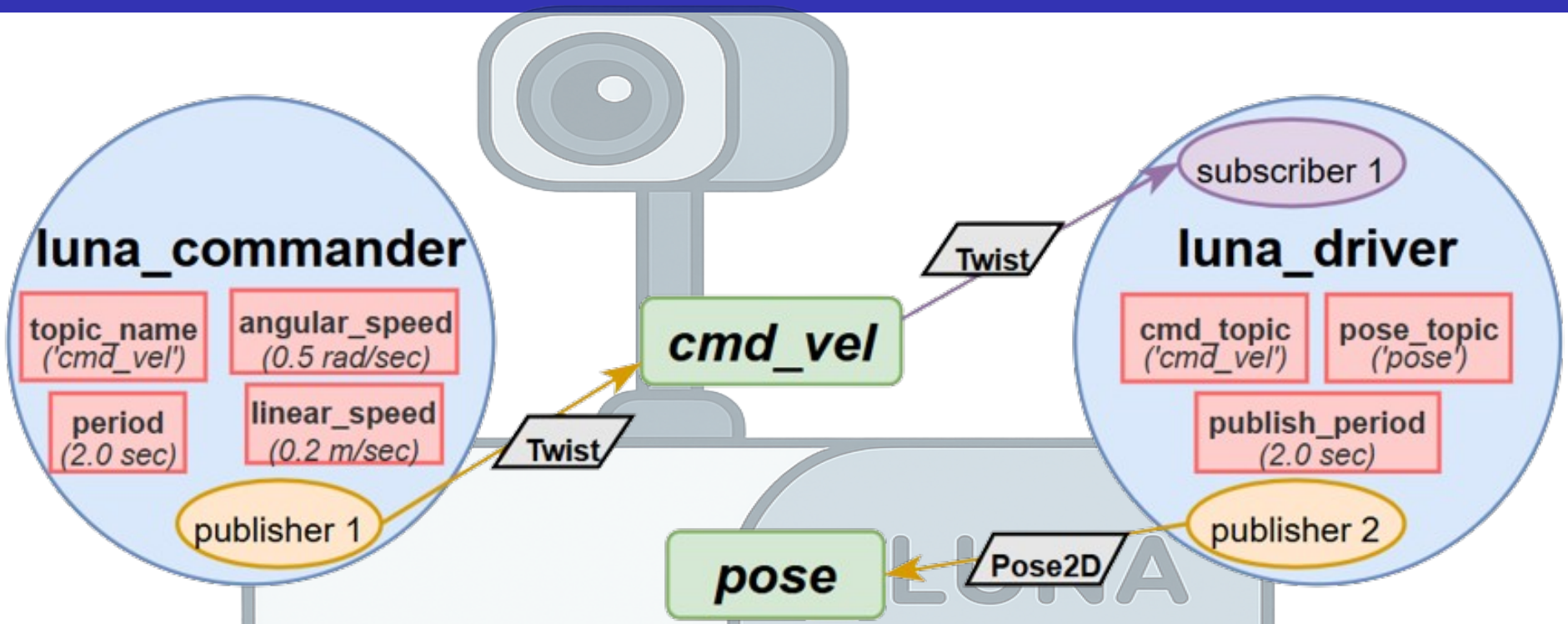
Call a service

`ros2 service call /<service_name> <service_type> <values>`

```
ros2 service call /luna_commander/get_parameters rcl_interfaces/srv/
    GetParameters "{names: ['linear_speed']}"
```

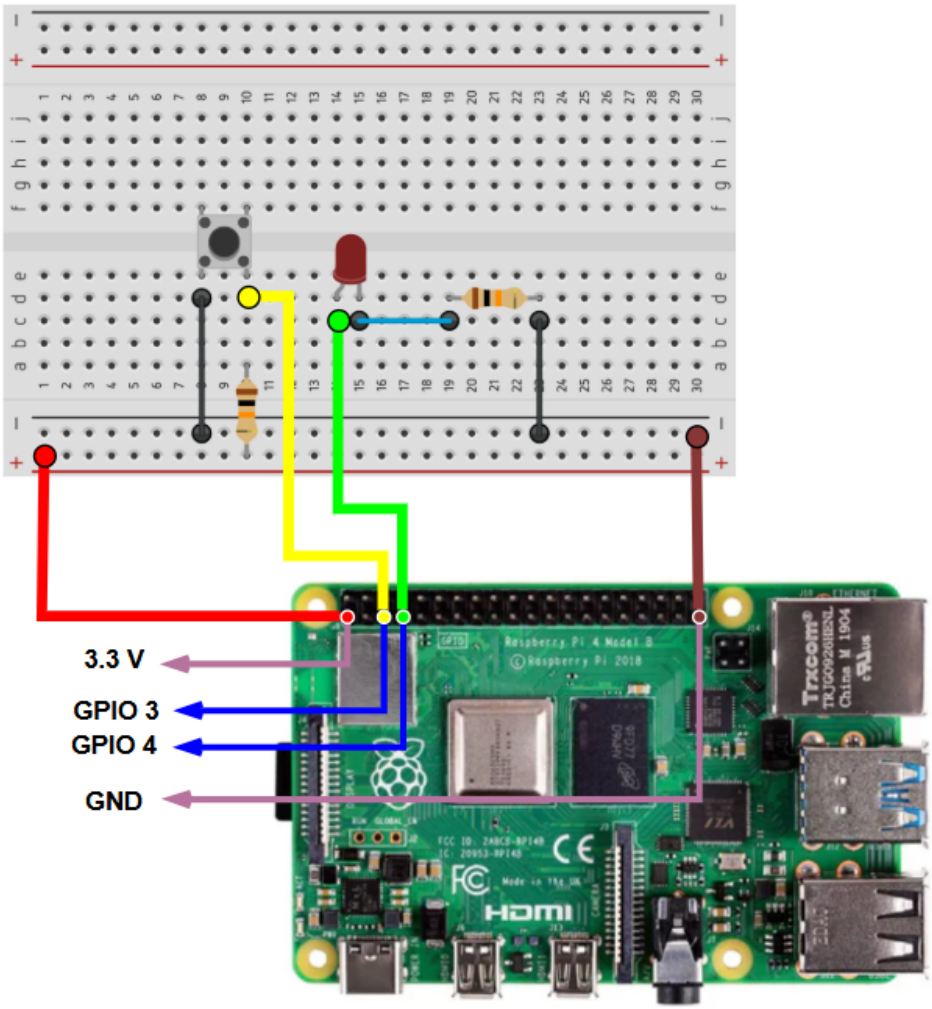
1. The command above is, in essence, what `ros2 param get /luna_commander linear_speed` does behind the scenes: the **parameters** of a node are served through **services**!
2. With the option `-r <Hz>` the call is repeated periodically.

Luna the Rover Exercise



We are going to simulate a robot called ***Luna***, moving in 2D space. The `luna_commander` node randomly generates velocity commands and publishes them as *`Twist`* messages. The `luna_driver` node subscribes to these commands, integrates them over time, and updates the robot's 2D pose, publishing it continuously as *`Pose2D`* messages. We can see that each node owns some key-value pairs, which we call **parameters** (see the Parameters section).

LED Exercise



RPi4 - LED circuit scheme

Let's test our ROS2 skills! Our goal: control a simple LED. We need:

- 1 RPi4
- 1 breadboard
- 1 LED
- 2 resistors (e.g. 10 kΩ)
- 1 Button
- connecting wires

Important!

Be very careful **not to short the 3.3V and GND pins!** At best, this might cause the RPi4 to reboot, but in the worst case, it could damage the board!

LED Exercise - Custom Interface: LedValue

For the LED we define **our own message**, inside a separate **interfaces package** (`ament_cmake`, see the *Interfaces Package* structure from before):

```
leds_rpi_pkg_interfaces/
  msg/LedValue.msg
  CMakeLists.txt
  package.xml
```

Our nodes depend on it, so `leds_rpi_pkg` lists `leds_rpi_pkg_interfaces` as a dependency in its `package.xml`.

msg/LedValue.msg

```
# 0 → LED is OFF
# 1 → LED is ON
uint32 led_value
```

Check it after building

```
ros2 interface show
leds_rpi_pkg_interfaces/msg/LedValue
```

Important!

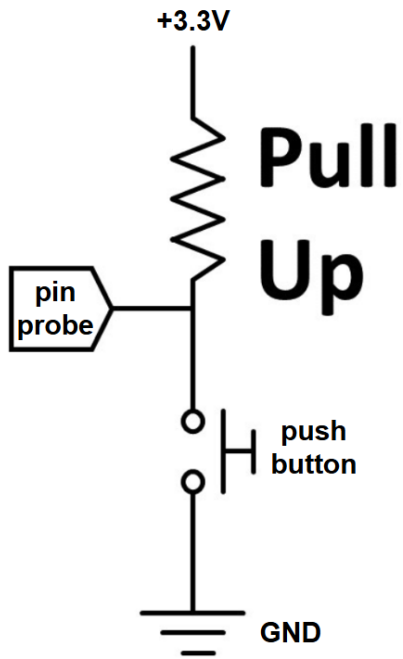
Build and source the workspace **before** using the new message in a node.

LED Exercise - Tasks

1. **Blink LED:** a *publisher node* sends **alternating 0 and 1** (our `LedValue` message) every 1 second over a topic. A *subscriber node* listens to it and its **callback blinks the LED**.
2. **Change blink rate: declare a parameter** for the timer's period and update the timer when it changes (it is only created at startup!). Try it on the circuit:
   `ros2 param set /blink_led_publisher blink_led_period 0.5`
3. **Add button:** a *second publisher node* reads a push button (pull-up resistor): LED **ON** while **pressed**, **OFF** when **released**. It publishes on the same topic, to the same subscriber.
4. **Launch files:** one for the blinking LED and one for the button, each starting its publisher together with the LED subscriber.

RPi.GPIO commands

```
RPi.GPIO.setmode(RPi.GPIO.BCM)
RPi.GPIO.setup(pin, RPi.GPIO.OUT)
RPi.GPIO.input(pin)          reads 0/1
RPi.GPIO.output(pin, value)
```



Push button with pull-up resistor: released → pin HIGH via the resistor, pressed → pin connected to GND, reads LOW.

References

- ROS Home
- ROS2 Documentation for all distros, here for Foxy
- rclpy library documentation
- Murilo's ROS2 Tutorial
- Kevin Wood ROS2 Youtube Tutorials
- Understanding Parameters in ROS2
- ROS2 YAML files for parameters
- Raspberry Pi 4 introduction and basics
- Control LED with RPi4 & ROS2: here_1 & here_2
- Elephant Robotics Documentation